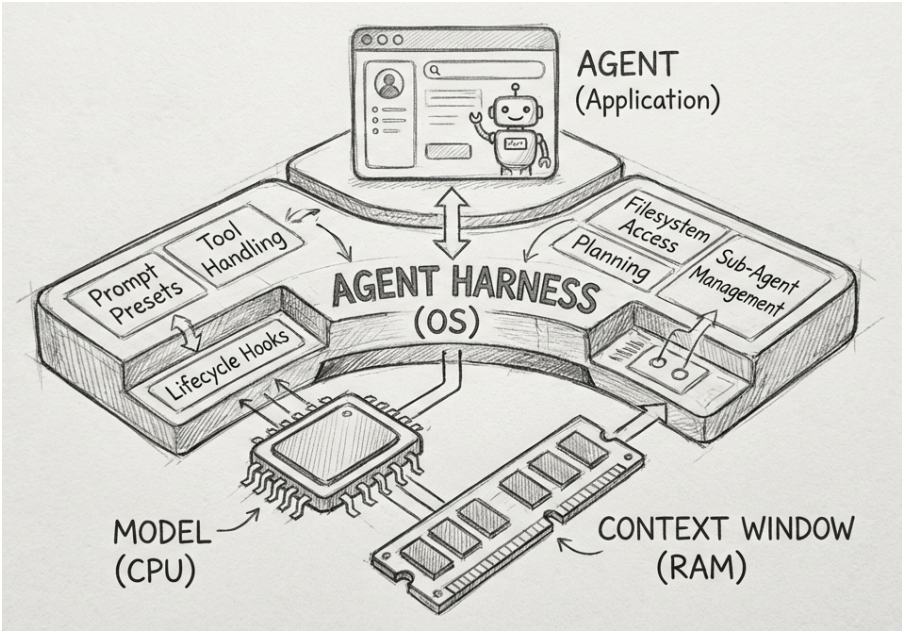


Harness Engineering 考核说明（2026年夏）

背景



Harness Engineering 考核

背景

Harness 是一种围绕 LLM 构建的外部框架，通过控制 LLM 的输入（Prompt 构造、记忆检索、上下文管理）和输出（解析、验证、多轮推理）来完成复杂任务。LLM 本身只能生成文本——它无法读文件、跑命令、改代码；Harness 的职责，是把模型输出中的“工具调用”截获、执行，再把结果回灌进对话历史，驱动模型继续推进，直到任务收敛。可以说，**模型是大脑，Harness 是身体**。

当前最具影响力的 AI 产品，很多本质上都是 Harness：**Claude Code**（Anthropic）、**Cursor**、**Codex CLI**（OpenAI）、**OpenCode**，乃至 **OpenClaw** 这类更自主的 Agent 产品，无一例外。一个被反复验证的现象是：同一个底层模型在不同 Harness 中的表现可以相差数十个百分点以上——在许多 Agent 基准上，把 Claude Opus 从最小脚手架搬进 Claude Code 的完整 Harness，分数能从相差几十个百分点。而这正是 Harness Engineering 的魅力所在：**同一个模型，设计更好的 Harness，能让它发挥出完全不同的能力水平**。

一个成熟的 Harness 通常由这几层构成：分层组装的系统提示词、工具集（Tools）的定义与描述、上下文工程（压缩、检索、按需加载）、子 Agent 编排（Subagent / Agent Teams 实现上下文隔离）、生命周期钩子（Hooks，把确定性逻辑下沉到代码层而非每次请求模型）、权限与沙箱（防注入、命令权限控制）、以及围绕“模型 → 工具调用 → 执行 → 回灌”的主循环。看似只有 60 行的循环骨架，复杂度全部沉淀在调优里——工具怎么切分、描述怎么写、上下文何时压缩、何时分发给子 Agent。

随着 LLM 能力边界不断扩展，Harness Engineering 已成为 AI 工程领域的核心技能之一——**模型层的差距正在收窄，Harness 层的工程能力，正在决定一个 AI 产品最终是“能 Demo”还是“能交付”**。从 Claude Agent SDK 到 Codex SDK，行业正在从“调用 LLM API”转向“构建在 Harness 之上”，Harness Engineering 的重要性也在迅速提升。

任务：限制输入窗口的 LLM 文本分类任务

大语言模型（LLM）的出现使得“无需训练、直接推理”成为可能——通过在 Prompt 中提供少量带标签示例（few-shot），LLM 可以在不更新任何参数的情况下完成新任务。如何利用 LLM 的语义理解性，在**不改变权重**的前提下，从少量带标签样本中快速“学习”并作出准确预测，用比较小参数的 LLM 在不损失很大性能的前提下替代传统的机器学习分类器，成为了一个重要的研究方向。

本次考核需要你设计一个基础的 Harness——一个含有外部记忆管理、预算控制与推理 Harness，使 LLM 在有限输入窗口（限制单轮输入 Token 数小于 2048）的文本分类任务上达到尽可能高的准确率。

系统首先会依次将带标签的训练样本喂给你的 Harness（`update`），你可以根据训练集更新 Harness 的记忆。训练流结束后，Harness 对无标签测试文本进行预测（`predict`）。**模型权重始终不变**，所有“学习”发生在 Harness 维护的外部状态中。最终成绩将由分类任务正确率决定。

数据集说明

本地 DEV 集为客服意图分类，共 77 类。正式评测将在**多个不同类型的任务**上进行，涵盖不同领域的文本分类与自然语言理解任务，以考查 Harness 的泛化能力，**因此请考生不要过拟合 DEV 集**。

所有数据集均为 JSONL 格式，每行一条样本。训练集和测试集字段相同，测试集的 `label` 字段仅用于本地评测，正式评测时考生无法访问：

代码块

```
1  {"text": "I no longer have my phone.", "label": "lost_or_stolen_phone"}
2  {"text": "My card is stuck in an ATM machine, how do I get it back quickly?",
   "label": "card_swallowed"}
```

- `text`：待分类的自然语言文本
- `label`：类别标签字符串，`predict()` 的返回值须与其**完全一致**（exact match）

每个任务保证：测试集中出现的所有标签均在对应训练集中出现过。

模型说明

整个运行代码，均采用 OpenAI Compatible（OpenAI 兼容）API 的风格进行代码的调用。考试过程，LLM API 需要考生自备（可以用公开平台的 API 服务，也可以自行用 sglang/vllm 进行部署）。为最小化考生成本，评分所用的模型为**Qwen3-8B (Instruct)**，且**不开思考模式**。

文件说明

代码块

1	<code>solution.py</code>	← 你唯一需要编辑的文件
2	<code>harness_base.py</code>	← Harness 基类（不可修改）
3	<code>llm_client.py</code>	← 配置你的 LLM API（修改顶部三行参数即可）
4	<code>run.py</code>	← 本地调试脚本（默认 4 轮取均值）
5	<code>data/</code>	
6	<code>train_dev.jsonl</code>	← DEV 训练集（231 条，77 类，每类 3 条）
7	<code>test_dev.jsonl</code>	← DEV 验证集（539 条，DEV 集以及最终任务集保证 test 集中出现的标签都在 train 集中出现过）
8	<code>tokenizer/</code>	← 本地 tokenizer（用于精确 token 计数）

快速开始

1. 安装依赖

代码块

```
1 pip install -r requirements.txt
```

2. 配置 LLM API

编辑 `llm_client.py` 顶部三行，填入你的 API 信息：

代码块

```
1 BASE_URL = "http://your-endpoint/v1"
2 API_KEY  = "your-api-key"
3 MODEL    = "your-model-name"
```

3. 实现你的 Harness

编辑 `solution.py` 中的 `MyHarness` 类，实现 `update` 和 `predict` 方法。

4. 本地测试

代码块

1	<code>python run.py</code>	# 默认设置，与最终评测参数一致
2	<code>python run.py --runs 1</code>	# 快速单轮测试（最终评测将取 4 轮均值）
3	<code>python run.py --workers 50</code>	# 调整 LLM 并发数，防止因超时等导致错误

接口说明

```

1 class MyHarness(Harness):
2     def __init__(self, call_llm, count_tokens, count_messages_tokens,
3         max_prompt_tokens: int):
4         super().__init__(call_llm, count_tokens, count_messages_tokens,
5             max_prompt_tokens)
6
7     def update(self, text: str, label: str) -> None:
8         """接收一条带标签的训练样本，更新内部记忆"""
9
10    def predict(self, text: str) -> str:
11        """对文本预测标签，返回标签字符串"""

```

基类提供以下注入接口：

属性	类型签名	说明
<code>self.call_llm</code>	<code>(messages: list[dict]) -> str</code>	调用 LLM，输入 OpenAI 格式 messages，返回回复文本
<code>self.count_tokens</code>	<code>(text: str) -> int</code>	计算单段文本的 token 数
<code>self.count_messages_tokens</code>	<code>(messages: list[dict]) -> int</code>	计算 messages 列表的总 token 数（只计算 content 总和，不应用 chat_template，与判题器判断是否需要截断一致）
<code>self.max_prompt_tokens</code>	<code>int</code>	每次调用的 prompt token 上限（2048）
<code>self.memory</code>	<code>list[tuple[str, str]]</code>	存储 <code>(text, label)</code> 训练样本

token 管理：单次 `call_llm` 的 prompt 超过 `max_prompt_tokens` 时会被截断尾部并在 stderr 打印警告。建议调用前用 `count_messages_tokens` 预先检查，主动控制 prompt 长度。

提交规则

- Python 文件：考生只需提交一个代码文件 `solution.py`，其中必须包含 `MyHarness` 类的完整实现。
 - 只允许 import Python 标准库（`re`、`math`、`random`、`collections` 等）、`numpy` 和 `harness_base`
 - 禁止读写任何文件

- 禁止通过任何途径获取测试集标签（一经发现得分归零），禁止出现直接编码公开的相应测试集，并采用穷举法搜索官方正确答案（私有测试集也无法在开源数据中找到）；禁止任何情况的不正当的分与 Hack 行为。每个考生的代码均会经过内容复核，一经发现，该项考核按 0 分计算。
2. 探索报告：PDF文件，简易记录探索过程，包含不同 Harness 设计策略的尝试、效果和分析，作为主观分数的参考之一。

提交方式：（截止时间 **北京时间 5月9号 00:00**，期间可以多次提交，会自动覆盖先前的提交文件）

1. 进入链接：<https://send2me.cn/bLSuiHmE/StyTAqcKANGDLA>
2. 精确填写个人信息，包括报名号、姓名（由填写错误导致的得分缺失，后果自负！）
3. FAQ网站：<https://docs.qq.com/sheet/DUXRkd1BQcXJDWGp3?u=5965604e4f164981b50cfc104734afec>，考生如果有需要向考官提问的问题，可以在该页面的“Question”列提出，考官会尽快给出解答。注意问题对所有人可见。

评分标准

本项考核的分数由两部分组成：

1. 客观得分（占总分80%）
 - 所有考生在私有集(每个任务的训练集和测试集格式与考生的 DEV 集完全相同，考生无法获取)上的加权平均准确率性能进行排名并赋分
 - 私有测试集包含以下任务（每个任务的格式均和 DEV 集完全相同）：
 - 与 DEV 集标签一致，文本不同的分类任务。（注：该部分测试集会含有较少比例的 Prompt Injection 样本，请注意 Harness 的安全性设计）
 - OOD 任务：若干个其他领域文本分类任务，内容、标签以及标签数量和 DEV 集完全不同。保证 test 集中出现的标签都在对应 train 集中出现过
 - 复杂自然语言选择题任务：格式一致，但文本变为自然语言选择题，标签变为选项（如 A/B/C/D），因此请考生不要针对文本分类任务设计过于特殊化的方案（如不使用 LLM 而设计了某种传统机器学习分类器），以免在该类任务上失去得分。
 - 最终得分会在多个任务正确率上加权计算得到
 - 测试统一使用Qwen3-8B (Instruct) 非思考模式模型，测试跑分的代码可以详见给定的 run.py
 - 每个任务会进行多次采样（默认4次）取平均指标以保证结果稳定性
2. 提示词主观评价得分（占总分20%）
 - 由专家老师基于指定评价准则进行评分
 - 评价内容包括 Harness 设计的创新性、合理性、可解释性等

注：正式评测时，会限制考生的任务执行时间（正常 Harness 设计不会超时），请考生不要进行恶意的无限轮调用 LLM 或用死循环卡住评测系统等行为，一经发现，该项考核按0分计算。

